A Type Theory beyond Animae

Michele Riva

University of Manchester

j.w.w. Denis-Charles Cisinski and Tashi Walde

December 22, 2025

MOTIVATION

∞-CATEGORY THEORY:

- Doing category theory "homotopically".
- Get rid of strict equality of maps, replaced by homotopy.
- Foundations of mathematics alternative to set theory.

MOTIVATION

∞-CATEGORY THEORY:

- Doing category theory "homotopically".
- Get rid of strict equality of maps, replaced by homotopy.
- Foundations of mathematics alternative to set theory.

PROBLEMS:

- Lack of unified foundations.
- Set-theoretical models against the spirit of homotopy.
- Computationally-heavy proofs.

GOAL: Axiomatising ∞-category theory within a type theory.

TYPE THEORY

TYPE THEORY: Derivation rules among **judgements**:

$\vdash A$ type	"A is a type"
$\vdash a : A$	"a is a term of type A"
$\vdash A \equiv B$ type	"A and B are equal types"
$\vdash a \equiv b : A$	"a and b are equal terms"

TYPE THEORY

TYPE THEORY: Derivation rules among **judgements**:

$$x : X \vdash A(x) \text{ type} \qquad \text{“}A \text{ is a type...}$$
$$x : X \vdash a(x) : A(x) \qquad \text{“}a \text{ is a term of type } A\text{...}$$
$$x : X \vdash A(x) \equiv B(x) \text{ type} \qquad \text{“}A \text{ and } B \text{ are equal types...}$$
$$x : X \vdash a(x) \equiv b(x) : A(x) \qquad \text{“}a \text{ and } b \text{ are equal terms...}$$

in **context** $x : X$”.

TYPE THEORY

TYPE THEORY: Derivation rules among **judgements**:

$x : X, y : Y(x) \vdash A(x,y)$ type — "A is a type...

$x : X, y : Y(x) \vdash a(x,y) : A(x,y)$ — "a is a term of type A...

$x : X, y : Y(x) \vdash A(x,y) \equiv B(x,y)$ type — "A and B are equal types...

$x : X, y : Y(x) \vdash a(x,y) \equiv b(x,y) : A(x,y)$ — "a and b are equal terms...

... in **context** $x : X, y : Y(x)$".

TYPE THEORY

TYPE THEORY: Derivation rules among **judgements**:

$x : X, y : Y(x) \vdash A(x,y)$ type — "A is a type...

$x : X, y : Y(x) \vdash a(x,y) : A(x,y)$ — "a is a term of type A...

$x : X, y : Y(x) \vdash A(x,y) \equiv B(x,y)$ type — "A and B are equal types...

$x : X, y : Y(x) \vdash a(x,y) \equiv b(x,y) : A(x,y)$ — "a and b are equal terms...

... in **context** $x : X, y : Y(x)$".

TYPES AS STATEMENTS:

- Statements about types and terms are formed as types.
- Terms of these statement types are the proofs.

TYPE THEORY

TYPE THEORY: Derivation rules among **judgements**:

$x : X, y : Y(x) \vdash A(x,y)$ type — "A is a type...

$x : X, y : Y(x) \vdash a(x,y) : A(x,y)$ — "a is a term of type A...

$x : X, y : Y(x) \vdash A(x,y) \equiv B(x,y)$ type — "A and B are equal types...

$x : X, y : Y(x) \vdash a(x,y) \equiv b(x,y) : A(x,y)$ — "a and b are equal terms...

... in **context** $x : X, y : Y(x)$".

TYPES AS STATEMENTS:

- Statements about types and terms are formed as types.
- Terms of these statement types are the proofs.

Identity type: $x : A,\ y : A \vdash \mathrm{Id}_A(x,y)$ type is "x is equal to y".

Terms $\alpha : \mathrm{Id}_A(x,y)$ give an internal notion of equality: **homotopy**.

HOMOTOPY TYPE THEORY

HoTT: A type theory that describes types as ∞-groupoids:

SYNTAX

- $\vdash B$ type
- $\vdash b : B$

INTUITION

- B ∞-groupoid
- $b : * \to B$.

HOMOTOPY TYPE THEORY

HoTT: A type theory that describes types as ∞-groupoids:

SYNTAX

- $\vdash B$ type
- $\vdash b : B$
- $x : A \vdash B(x)$ type
- $x : A \vdash f(x) : B(x)$

INTUITION

- B ∞-groupoid
- $b : * \to B$
- $B \twoheadrightarrow A$ isofibration
- f section of $B \twoheadrightarrow A$

HOMOTOPY TYPE THEORY

HoTT: A type theory that describes types as ∞-groupoids:

SYNTAX

- $\vdash B$ type
- $\vdash b : B$
- $x : A \vdash B(x)$ type
- $x : A \vdash f(x) : B(x)$
- $\vdash A, B$ type $\rightsquigarrow x : A \vdash B$ type (weakening of B)
- $x : A \vdash f(x) : B$

INTUITION

- B ∞-groupoid
- $b : * \to B$
- $B \twoheadrightarrow A$ isofibration
- f section of $B \twoheadrightarrow A$
- Given A, B ∞-groupoids we have $B \times A \twoheadrightarrow A$
- A section of $B \times A \twoheadrightarrow A$, that is a functor $A \to B$

FUNCTORIALITY OF JUDGEMENTS

FUNCTORIALITY: Judgements are "functorial":

$$x : A \vdash f(x) : B(x).$$

- This is mapping any $\vdash a : A$ into a $\vdash f(a) : B(a)$.

This gives a proof of a predicate about all points $a : A$ in A.

FUNCTORIALITY OF JUDGEMENTS

FUNCTORIALITY: Judgements are "functorial":

$$x : A \vdash f(x) : B(x).$$

- This is not just mapping any $\vdash a : A$ into a $\vdash f(a) : B(a)$.
- For any $\alpha : S \to A$ we can assign $y : S \vdash f(\alpha(y)) : B(\alpha(y))$.

This gives a proof of a predicate about **all maps** into A.

FUNCTORIALITY OF JUDGEMENTS

FUNCTORIALITY: Judgements are "functorial":

$$x : A \vdash f(x) : B(x).$$

- This is not just mapping any $\vdash a : A$ into a $\vdash f(a) : B(a)$.
- For any $\alpha : S \to A$ we can assign $y : S \vdash f(\alpha(y)) : B(\alpha(y))$.

This gives a proof of a predicate about **all maps** into A.

EXAMPLE: Let $\vdash A, C$ type and assume we can derive a rule

$$\frac{\vdash a : A}{\vdash f(a) : C}. \qquad \text{We cannot derive} \qquad x : A \vdash f(x) : C.$$

Pointwise rules are weaker than judgements a priori.

FUNCTORIALITY OF JUDGEMENTS

FUNCTORIALITY: Judgements are "functorial":

$$x : A \vdash f(x) : B(x).$$

- This is not just mapping any $\vdash a : A$ into a $\vdash f(a) : B(a)$.
- For any $\alpha : S \to A$ we can assign $y : S \vdash f(\alpha(y)) : B(\alpha(y))$.

This gives a proof of a predicate about **all maps** into A.

EXAMPLE: Let $\vdash A, C$ type and assume we can derive a rule

$$\forall \Gamma \ \frac{\Gamma \vdash a : A}{\Gamma \vdash f(a) : C}. \qquad \text{We can derive} \qquad x : A \vdash f(x) : C,$$

by applying the rule to the universal term: $\dfrac{x : A \vdash x : A}{x : A \vdash f(x) : C}$.

LIMITS OF HOMOTOPY TYPE THEORY

HOTT

The rules are the same in all contexts.

- Every pointwise assignment/property becomes functorial.
- If an embedding of types has a pointwise section, then it is an equivalence.
- Types are ”statements”: they consist of points and homotopies.

MEANING

The theory of isofibrations of groupoids over a fixed Γ is the theory of groupoids.

LIMITS OF HOMOTOPY TYPE THEORY

HOTT

The rules are the same in all contexts.

- Every pointwise assignment/property becomes functorial.
- If an embedding of types has a pointwise section, then it is an equivalence.
- Types are ”statements”: they consist of points and homotopies.

CATEGORY THEORY

The rules are the same only over some contexts.

- Assignments on objects between categories do not lift to functors.
- Want to access properties over the objects of a category only.
- Categories can be full “theories”, category theory describes itself.

GOALS

We want to construct a type theory where:

(1) The rules are not the same in all contexts, to get
- Internal Homs (Π-types) only in some contexts.
- statements expressing objectwise, non-functorial properties.

GOALS

We want to construct a type theory where:

(1) The rules are not the same in all contexts, to get
- Internal Homs (Π-types) only in some contexts.
- statements expressing objectwise, non-functorial properties.

(2) We have directed, non-invertible, morphisms.

GOALS

We want to construct a type theory where:

(1) The rules are not the same in all contexts, to get
- Internal Homs (Π-types) only in some contexts.
- statements expressing objectwise, non-functorial properties.

(2) We have directed, non-invertible, morphisms.

(3) Universe types are not just "statements": they describe the full theory of types and maps between them.

This will make the types richer, but harder to manipulate. This will allow us to interpret types as categories.

ANIMAE

STRATEGY: We introduce a class of special contexts, **"animae"**:

$$\vdash \Gamma \text{ anima}.$$

- Formulate the theory so that the rules over any $\vdash \Gamma$ anima are always the same as in empty context.
- Given $\vdash \Gamma$ anima, and $\Gamma \vdash A$ type, we say that A is a **"groupoid"** if $\vdash \Gamma, x : A$ anima.
- If A is a groupoid, we can manipulate A as in HoTT: everything about the points $\Gamma \vdash a : A$ will be lifted to context $\Gamma, x : A$ by structure of the theory $\rightsquigarrow$ groupoids are the **"statements"**.

This distinction, i.e. the presence of non-groupoid types, is a new feature of the theory which can go beyond describing animae.

RULES IN COMMON WITH HOTT

We introduced an asymmetry in the theory, allowing us to formulate some rules only over animae. However, some rules of HoTT will still be the same, introduced in arbitrary contexts:

- Intensional identity types (we have a path object for isofibration of categories in every context). In particular we have a notion of equivalence between types.
- Σ-types (isofibrations of categories compose).
- Terminal type $\Gamma \vdash *$ type (there is a terminal category).

The usual rules apply to these types, and express the fact that we have a "tribe" structure on categories given by isofibrations. Let us now see the first differences.

TYPES OF MAPS

A map of types $\Gamma, x : A \vdash f(x) : B$ should be a functor $A \to B$ (over Γ).

- In HoTT, given $\Gamma, x : A \vdash B(x)$ type, we can form the Π-type $\Gamma \vdash \Pi_{x:A} B(x)$ type. This is the existence of a right adjoint to the pullback along an isofibration of groupoids.
  - In particular, get internal Homs $\Gamma \vdash X \to Y$ type.

Types of maps

A map of types $\Gamma, x : A \vdash f(x) : B$ should be a functor $A \to B$ (over Γ).

- In HoTT, given $\Gamma, x : A \vdash B(x)$ type, we can form the Π-type $\Gamma \vdash \Pi_{x:A} B(x)$ type. This is the existence of a right adjoint to the pullback along an isofibration of groupoids.
  - In particular, get internal Homs $\Gamma \vdash X \to Y$ type.
- In this theory, we can form types of maps over animae, but not in general: not all isofibrations are exponentiable. Given $\vdash \Gamma$ anima and $\Gamma \vdash X, Y$ type we can either:
  (1) introduce functor categories $\mathrm{Fun}(X, Y)$ as right adjoints, which are **not** groupoids.
  (2) Introduce a **groupoid** of maps $\mathrm{Map}(X, Y)$.

  What is the right pick to start?

TYPE OF MAPS FOR YONEDA

Consider $f : X \to Y$ map of types in our theory.

- The existence of non-groupoid types implies that checking f is an equivalence is harder than in HoTT in general.
- In ordinary category theory we often rely on the Yoneda Lemma.

TYPE OF MAPS FOR YONEDA

Consider $f : X \to Y$ map of types in our theory.

- The existence of non-groupoid types implies that checking f is an equivalence is harder than in HoTT in general.
- In ordinary category theory we often rely on the Yoneda Lemma.
- If we had a groupoid Map, we could recover it as follows: to prove f is an equivalence, we check instead that for every Z

$$f_* : \mathrm{Map}(Z, X) \to \mathrm{Map}(Z, Y)$$

  is an equivalence of groupoids. These equivalences are simpler.
- Note that taking Fun instead of Map would not work: for $Z = *$ the map f_* is f.

  $\Rightarrow$ we introduce the Map constructor as soon as possible.

Dependent mapping spaces

Similar rules to the usual Π-type,

$$\frac{\vdash \Gamma \text{ anima} \quad \Gamma, a : A \vdash B(a) \text{ type}}{\Gamma \vdash \mathrm{Sec}_{a:A} B(a) \text{ type}}$$

$$\frac{\vdash \Gamma \text{ anima} \quad \Gamma \vdash g(a) : \mathrm{Sec}_{a:A} B(a)}{\Gamma, a : A \vdash \mathrm{ev}_g(a) : B(a) \text{ type}} \qquad \frac{\vdash \Gamma \text{ anima} \quad \Gamma, a : A \vdash f(a) : B(a)}{\Gamma \vdash \lambda a.f(a) : \mathrm{Sec}_{a:A} B(a) \text{ type}}$$

+ $\lambda a.\mathrm{ev}_g(a) = g$ and $\mathrm{ev}_{\lambda x.g(x)}(a) = g(a)$, determining the terms of $\mathrm{Sec}_{a:A} B(a)$ as the dependent functions from A to B,
+ a function extensionality rule determining the Id-type of Sec.

Dependent mapping spaces

Similar rules to the usual Π-type, except for $(*)$:

$$\frac{\vdash \Gamma \text{ anima} \quad \Gamma, a : A \vdash B(a) \text{ type}}{\Gamma \vdash \mathrm{Sec}_{a:A} B(a) \text{ type}} \qquad \frac{\vdash \Gamma \text{ anima} \quad \Gamma, a : A \vdash B(a) \text{ type}}{\Gamma \vdash \mathrm{Sec}_{a:A} B(a) \text{ groupoid}} \ (*)$$

$$\frac{\vdash \Gamma \text{ anima} \quad \Gamma \vdash g(a) : \mathrm{Sec}_{a:A} B(a)}{\Gamma, a : A \vdash \mathrm{ev}_g(a) : B(a) \text{ type}} \qquad \frac{\vdash \Gamma \text{ anima} \quad \Gamma, a : A \vdash f(a) : B(a)}{\Gamma \vdash \lambda a.f(a) : \mathrm{Sec}_{a:A} B(a) \text{ type}}$$

+ $\lambda a.\mathrm{ev}_g(a) = g$ and $\mathrm{ev}_{\lambda x.g(x)}(a) = g(a)$, determining the terms of $\mathrm{Sec}_{a:A} B(a)$ as the dependent functions from A to B,
+ a function extensionality rule determining the Id-type of Sec.

Mapping space and groupoid core

Let us consider two particular cases:

- In the non-dependent case, we can form

$$\frac{\vdash \Gamma \text{ anima} \qquad \Gamma \vdash A, C \text{ type}}{\Gamma \vdash \mathrm{Sec}_{a:A} C \equiv: \mathrm{Map}(A, C) \text{ type}}$$

  whose terms are maps $A \to C$.

Mapping space and groupoid core

Let us consider two particular cases:

- In the non-dependent case, we can form

$$\frac{\vdash \Gamma \text{ anima} \quad \Gamma, t : T \vdash A(t), C(t) \text{ type}}{\Gamma \vdash \mathrm{Sec}_{t:T, a:A(t)} \equiv: \mathrm{Map}_{t:T}(A(t), C(t)) \text{ type}}$$

  whose terms are maps $A \to C$.

Mapping space and groupoid core

Let us consider two particular cases:

- In the non-dependent case, we can form

$$\frac{\vdash \Gamma \text{ anima} \quad \Gamma, t : T \vdash A(t), C(t) \text{ type}}{\Gamma \vdash \mathrm{Sec}_{t:T, a:A(t)} \equiv: \mathrm{Map}_{t:T}(A(t), C(t)) \text{ type}}$$

  whose terms are maps $A \to C$.
- A case of particular interest is when we form

$$\frac{\vdash \Gamma \text{ anima} \quad \Gamma \vdash A \text{ type}}{\Gamma \vdash \mathrm{Map}(*, A) \equiv: A^{\simeq} \text{ type}}.$$

  We call $A^{\simeq}$ the **groupoid core** of A. By the rules, $A^{\simeq}$ is the universal groupoid mapping into A.

APPLICATION 1: Π-TYPES

We can now axiomatise internal Homs using mapping spaces

$$\frac{\vdash \Gamma \text{ anima} \quad \Gamma, a : A \vdash B(a) \text{ type}}{\Gamma \vdash \Pi_{a:A} B(a) \text{ type}} \qquad \frac{\vdash \Gamma \text{ anima} \quad \Gamma \vdash g(a) : \Pi_{a:A} B(a)}{\Gamma, a : A \vdash \text{ev}_g(a) : B(a) \text{ type}}$$

$$\frac{\vdash \Gamma \text{ anima} \quad \Gamma, a : A \vdash B(a) \text{ type} \quad \Gamma \vdash C \text{ type}}{\Gamma \vdash \Pi\text{-as-radj}_{B/A} : \text{isEquiv}(\text{Map}(C, \Pi_{a:A} B(a)) \xrightarrow{\text{ev}_*} \text{Map}_{a:A}(C, B(a)))}.$$

Application 1: Π-types

We can now axiomatise internal Homs using mapping spaces

$$\frac{\vdash \Gamma \text{ anima} \quad \Gamma, a : A \vdash B(a) \text{ type}}{\Gamma \vdash \Pi_{a:A} B(a) \text{ type}} \qquad \frac{\vdash \Gamma \text{ anima} \quad \Gamma \vdash g(a) : \Pi_{a:A} B(a)}{\Gamma, a : A \vdash \mathrm{ev}_g(a) : B(a) \text{ type}}$$

$$\frac{\vdash \Gamma \text{ anima} \quad \Gamma, a : A \vdash B(a) \text{ type} \quad \Gamma \vdash C \text{ type}}{\Gamma \vdash \Pi\text{-as-radj}_{B/A} : \mathrm{isEquiv}(\mathrm{Map}(C, \Pi_{a:A} B(a)) \xrightarrow{\mathrm{ev}_*} \mathrm{Map}_{a:A}(C, B(a)))}.$$

- $\Pi_{a:A} B(a)$ is introduced as a right adjoint to the pullback.
- The terms are dependent functions from A to B, like $\mathrm{Sec}_{a:A} B(a)$, but $\Pi_{a:A} B(a)$ is not a groupoid.
- **Theorem**: The ordinary function extensionality holds:

$$\mathrm{Id}_{\Pi_{a:A} B(a)}(f, g) \simeq \Pi_{a:A} \mathrm{Id}_B(\mathrm{ev}_f(a), \mathrm{ev}_g(a)).$$

When B does not depend on A we define $\Gamma \vdash \mathrm{Fun}(A, B) :\equiv \Pi_{a:A} B$ type.

Application 2: Objectwise statements

- We saw that judgements are automatically functorial.
- As opposed to HoTT, pointwise rules do not become functorial judgements: what we say about points in $\vdash A$ type becomes true only for maps from groupoids into A.

Given $\vdash \Gamma$ anima and $\Gamma \vdash A, B$ type

$$\frac{\Gamma \vdash a : A}{\Gamma \vdash f(a) : B} \quad \not\rightsquigarrow \quad \Gamma, a : A \vdash f(a) : B,$$

APPLICATION 2: OBJECTWISE STATEMENTS

- We saw that judgements are automatically functorial.
- As opposed to HoTT, pointwise rules do not become functorial judgements: what we say about points in $\vdash A$ type becomes true only for maps from groupoids into A.

Given $\vdash \Gamma$ anima and $\Gamma \vdash A, B$ type

$$\frac{\Gamma \vdash a : A}{\Gamma \vdash f(a) : B} \quad \not\rightsquigarrow \quad \Gamma, a : A \vdash f(a) : B,$$

but we know that we have the same rule over any anima, and:

$$\frac{\vdash \Gamma, t : T \text{ anima} \quad \Gamma, t : T \vdash a(t) : A}{\Gamma, t : T \vdash f(a(t)) : B} \quad \overset{T = A^{\simeq}}{\rightsquigarrow} \quad \Gamma, a : A^{\simeq} \vdash f(a) : B$$

We call this result an objectwise statement in A: judgement that only varies over maps from groupoids into A (its "objects").

INTERNAL DIRECTED MORPHISMS

- In HoTT, internal morphisms are homotopies of points.
- Here, we introduce internal morphism via a 1-simplex type

$$\frac{\vdash \Gamma \text{ ctx}}{\Gamma \vdash \Delta^1 \text{ type} \qquad \Gamma \vdash 0, 1 : \Delta^1}.$$

- We can construct a 2-simplex via $\Gamma \vdash \Delta^2 :\equiv \mathrm{Fun}(\Delta^1, \Delta^1)$.

For $\vdash \Gamma$ anima and $\Gamma \vdash A$ type we say:

(1) Arrows in A are maps $f : \Delta^1 \to A$, displayed as $f : a \to b$.

(2) Commutative triangles in A are $t : \Delta^2 \to A$, displayed as

$$\begin{array}{ccc} a & \xrightarrow{f} & b \\ & \searrow^{h} & \downarrow{\scriptstyle g} \\ & & c \end{array}.$$

h: a composition of f and g.

EXISTENCE AND UNIQUENESS OF COMPOSITION

AXIOM: The following functor is an equivalence of types

$$\mathrm{Seg}^2 : \mathrm{Fun}(\Delta^2, A) \to \mathrm{Fun}(\Delta^1, A) \times_A \mathrm{Fun}(\Delta^1, A)$$

$$\begin{array}{ccc} a & \xrightarrow{f} & b \\ & \searrow^{h} & \downarrow{\scriptstyle g} \\ & & c \end{array} \quad \longmapsto \quad \begin{array}{ccc} a & \xrightarrow{f} & b \\ & & \downarrow{\scriptstyle g} \\ & & c \end{array}$$

EXISTENCE AND UNIQUENESS OF COMPOSITION

AXIOM: The following functor is an equivalence of types

$$\mathrm{Seg}^2 : \mathrm{Fun}(\Delta^2, A) \to \mathrm{Fun}(\Delta^1, A) \times_A \mathrm{Fun}(\Delta^1, A)$$

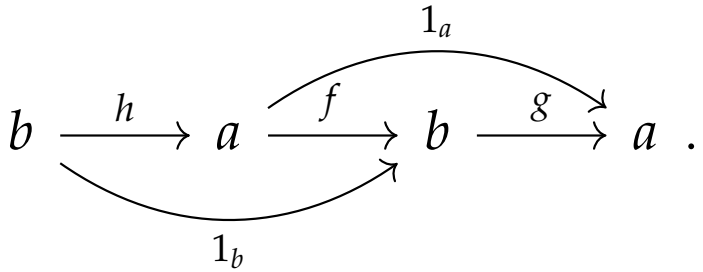
$$\begin{array}{ccc} a & \xrightarrow{f} & b \\ & \searrow^{h} & \downarrow{\scriptstyle g} \\ & & c \end{array} \quad \longmapsto \quad \begin{array}{ccc} a & \xrightarrow{f} & b \\ & & \downarrow{\scriptstyle g} \\ & & c \end{array}$$

$\Rightarrow$ Existence and uniqueness of composition of morphisms.

PROPOSITION: Composition is associative and unital with respect to identities $1_a : \Delta^1 \xrightarrow{!} \Delta^0 \xrightarrow{a} A$.

ISOMORPHISMS

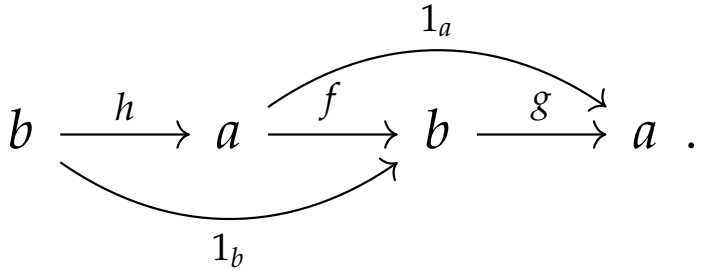
Given $a, b : A$, an **isomorphism** $f : a \cong b$ is the datum of



Construct $\Gamma, a : A, b : A \vdash \mathrm{Iso}_A(a, b)$ type of isomorphisms.

ISOMORPHISMS

Given $a, b : A$, an **isomorphism** $f : a \cong b$ is the datum of



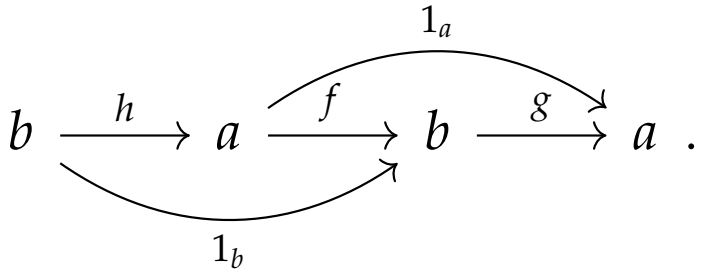
Construct $\Gamma, a : A, b : A \vdash \mathrm{Iso}_A(a, b)$ type of isomorphisms.

PROBLEM: Two ways to identify objects $a, b : A$:

- Providing a homotopy $\alpha : \mathrm{Id}_A(x, y)$.
- Providing an isomorphism $\beta : \mathrm{Iso}_A(a, b)$.

ISOMORPHISMS

Given $a, b : A$, an **isomorphism** $f : a \cong b$ is the datum of



Construct $\Gamma, a : A, b : A \vdash \mathrm{Iso}_A(a, b)$ type of isomorphisms.

PROBLEM: Two ways to identify objects $a, b : A$:

- Providing a homotopy $\alpha : \mathrm{Id}_A(x, y)$.
- Providing an isomorphism $\beta : \mathrm{Iso}_A(a, b)$.

AXIOM: $\mathrm{Iso}_A(x, y) \xrightarrow{\sim} \mathrm{Id}_A(x, y)$ in context $\Gamma, x : A, y : A$.

IDENTIFYING GROUPOIDS

- Up to now we started with an abstract notion of anima/groupoid.
- Having access to Δ^1 and Fun, we can now characterise groupoids via a checkable property.

Given $\vdash \Gamma$ anima and $\Gamma \vdash A$ type we can construct a $\Gamma \vdash \mathrm{isGpd}(A)$ type whose terms are proofs of $A \to \mathrm{Fun}(\Delta^1, A)$, $a \mapsto 1_a$, being an equivalence. We axiomatise

$$\frac{\Gamma \vdash \alpha : \mathrm{isGpd}(A)}{\Gamma \vdash A \text{ groupoid}} \qquad \frac{\Gamma \vdash A \text{ groupoid}}{\Gamma \vdash \text{anima-groupoid}_A : \mathrm{isGpd}(A)}.$$

In particular, groupoids are stable under equivalences.

FUTURE WORK

A lot of category theory must still be formalised precisely, among which:

- Subcategories.
- Localisations.
- The theory of (co)cartesian fibrations and their exponentiability.
- Tools to deduce global conditions from objectwise ones (e.g. equivalence iff fully faithful and essentially surjective).
- Universe types and directed univalence.

A QUICK LOOK AT UNIVERSES

- In HoTT, universes do not contain all information of the theory: they miss the non-invertible functors of groupoids.
- Here, since we have non-groupoidal types, universes can contain the whole theory. Univalence will then be replaced by the straightening-unstraightening correspondence: a universe will be a type Cat for which

$$\mathrm{Fun}(A, \mathrm{Cat}) \simeq \mathrm{cocart}(A) \qquad \text{(up to smallness)}.$$

- If our types were 1-categories, this equivalence would only be of 2-categories, we could not stay within the language. Thus, here we are forced to go to ∞-categories.

References



Carlo Angiuli and Daniel Gratzer.
Principles of dependent type theory.
Lecture notes for courses at Indiana University and Aarhus University. Version from April, 12:2024, 2024.



Andrej Bauer, Jason Gross, Peter LeFanu Lumsdaine, Michael Shulman, Matthieu Sozeau, and Bas Spitters.
The hott library: a formalization of homotopy type theory in coq.
In *Proceedings of the 6th ACM SIGPLAN Conference on Certified Programs and Proofs*, pages 164–172, 2017.



Denis-Charles Cisinski, Bastiaan Cnossen, Kim Nguyen, and Tashi Walde.
Formalization of higher categories.
`https://sites.google.com/view/bastiaan-cnossen`.